CHMLTECH ONLINE JOURNAL · VOLUME 105, ISSUE 1 · RESEARCH ARTICLE

# Physical AI: Foundations, Architectures and the Simulation-to-Reality Gap

**Quak Foo Lee**
Chief Technology Officer, Chmltech Ltd.
Division of Artificial Intelligence and Machine Learning · Vancouver, BC, Canada

Published November 9, 2026  17 min read

ABSTRACT

Physical AI denotes learned decision-making systems that sense and act in the material world through sensors and actuators, in closed loop. This paper reviews its technical foundations for an engineering readership. It first defines the term and contrasts it with digital AI in the consequences of error, the scarcity of training data, real-time constraints and the cost of evaluation. It then describes the perception, planning and control stack, the representation of a robot policy as a conditional distribution over action sequences, and the timescales at which large models and low-level controllers operate. Reinforcement learning and imitation learning are compared, including the quadratic growth of error with task length in behavior cloning and its mitigation by action chunking and generative action models. Vision-language-action models from RT-1 to GR00T N1 are reviewed together with the pooled datasets on which they depend. Physics simulators and learned world models are discussed as sources of training experience, and the simulation-to-reality gap is formalized and related to its causes in dynamics, timing and perception. System identification, domain randomization, privileged learning, online adaptation and real-data fine-tuning are compared as remedies. The paper concludes that the methods of

physical AI are established, that simulation-only training succeeds where physics is rigid-body and objectives are simple, and that reliability over long tasks and under unfamiliar conditions is the principal remaining obstacle.

**Keywords:** physical AI; embodied AI; vision-language-action models; sim-to-real transfer; domain randomization; imitation learning; reinforcement learning; world models

## 1. Introduction

Artificial intelligence has, until recently, been most visible in systems whose inputs and outputs are digital: text, images, code and structured data. Physical AI refers to systems that close the loop through the material world. They sense their surroundings, decide what to do, and act through motors, grippers, wheels or valves, and the consequences of their actions return as new sensor readings. The phrase came into wide use in 2024 and 2025; one widely cited formulation describes it as AI equipped with sensors and actuators and argues that such systems must be trained digitally before they are deployed physically [1]. The underlying idea is older. Brooks argued in 1991 that intelligent behavior is best studied in systems that are situated in, and act directly upon, the real world rather than on symbolic abstractions of it [2].

What has changed is the technical route. For decades, robots were programmed: engineers wrote explicit models of the robot and its task and tuned controllers by hand. Learning-based methods were studied throughout, but surveys of the field emphasized how hard they were to apply on real hardware, where samples are expensive, state and action spaces are continuous and high-dimensional, and simulation models are imperfect [3]. Three developments since about 2017 have moved learning from the laboratory toward practice: reinforcement learning in massively parallel simulation, imitation learning from teleoperated demonstrations with expressive generative policies, and robot foundation models that inherit knowledge from vision-language models trained on web-scale data [4].

This paper reviews the foundations of physical AI for an engineering readership. Section 2 defines the term and contrasts it with digital AI. Section 3 describes the perception, planning and control stack and the timescales at which its layers operate. Section 4 covers the two main learning paradigms and the compounding-error problem. Section 5 reviews vision-language-action models and the data on which they are trained. Section 6 discusses simulators and learned world models. Section 7 treats the simulation-to-reality gap, which is

the central obstacle to training digitally and deploying physically, and Section 8 summarizes evaluation practice and open problems. Applications in process plants and factories, and questions of safety and economics, are treated in companion papers in this volume.

# 2. Definitions and Scope

## 2.1 What physical AI is

In this paper, physical AI denotes a learned decision-making system that (i) receives observations from physical sensors, (ii) produces actions that are executed by physical actuators, and (iii) operates in closed loop, so that each action influences later observations. Robots are the most familiar embodiment, including manipulator arms, mobile robots, legged machines, drones and humanoids, but the definition also covers autonomous vehicles and learned controllers for industrial equipment. The definition excludes systems that only perceive, such as a camera that classifies defects for a human to act upon, and systems that only recommend. Those systems are valuable, but their errors are filtered by a person before anything moves.

## 2.2 How physical AI differs from digital AI

The closed physical loop changes the engineering problem in four ways. First, errors have physical consequences: a wrong action can damage equipment or injure a person, and it cannot be retracted. Second, data are scarce. Language models learn from text that already exists on the internet, whereas robot actions must be recorded on robots; no comparable corpus of motor experience exists [4]. Third, the system must meet real-time deadlines, because the world does not pause while a model computes. Fourth, evaluation is slow, since every trial requires a robot, a scene and a reset. Table 1 summarizes the contrast. These differences explain why progress in physical AI has lagged progress in language and vision, and why techniques that reduce the need for real-world data dominate the field.

**Table 1.** Contrasts between digital AI and physical AI that shape system design.

| Aspect | Digital AI (e.g., language models) | Physical AI |
|---|---|---|
| Output | Text, images, code, recommendations | Motor commands executed on hardware |
| Consequence of an error | Usually reviewable and reversible | Immediate, physical, often irreversible |
| Training data | Web-scale corpora that already exist | Robot experience that must be collected or simulated |
| Timing | Latency affects convenience | Latency affects stability and safety |
| Evaluation | Automated benchmarks at scale | Physical trials, or simulation with a reality gap |
| Distribution shift | New topics or phrasing | New objects, lighting, wear, friction, layouts |

Compiled by the author from [3, 4].

# 3. The Perception, Planning and Control Stack

## 3.1 Modular and end-to-end designs

A classical robot is organized as a pipeline. Perception converts sensor data into a state estimate, such as object poses or a map; planning computes a trajectory that achieves the task; and control tracks that trajectory with feedback at the joints. Each module is engineered and verified separately, which makes failures traceable, but the interfaces between modules discard information, and each module depends on models that must be built by hand. End-to-end learning replaces part or all of the pipeline with a neural network policy trained to map observations to actions. Levine and co-authors showed that a single convolutional network could be trained to map raw camera images directly to motor torques, and that training perception and control jointly gave better performance than training them separately [5]. Current systems lie between the two extremes: a learned policy produces target poses or joint positions, and conventional controllers execute them.

## 3.2 The policy as a conditional distribution

A modern robot policy can be written as a conditional distribution over a short sequence of future actions, given recent observations and an instruction:

$$a_{t:t+H} \sim \pi_\theta(\,\cdot \mid o_{t-k:t},\, \ell) \tag{1}$$

where $o$ denotes observations (camera images and joint readings), $\ell$ is a language instruction or goal, $\theta$ are the network parameters, $k$ is the length of the observation history and $H$ is the action horizon. Predicting a chunk of $H$ actions at once, rather than one action per step, was introduced to reduce the effective number of decisions in a task and to cope with pauses and inconsistencies in human demonstrations [6]. Treating the policy as a distribution rather than a single-valued function matters because many tasks have several valid solutions, such as passing an obstacle on the left or on the right; averaging between them gives an invalid action.

## 3.3 Timescales

The layers of the stack operate at very different rates, and the rate at which a large model can run limits what it can control directly. The largest RT-2 model, with 55 billion parameters, ran at 1 to 3 Hz and had to be served from a multi-accelerator cloud service; a 5-billion-parameter variant reached about 5 Hz [7]. OpenVLA, with 7 billion parameters, runs at about 6 Hz on a single high-end graphics processor, and its authors note that this is too slow for bimanual systems controlled at 50 Hz [8]. Two designs address the mismatch. The first keeps a large vision-language backbone but replaces token-by-token decoding with a continuous action generator; $\pi_0$ uses flow matching to produce action chunks at up to 50 Hz [9]. The second separates a slow reasoning module from a fast action module. GR00T N1 runs its vision-language module at 10 Hz and a diffusion-transformer action module at 120 Hz, an arrangement its authors call System 2 and System 1 [10]. Below the learned policy, joint-level servo loops run faster again and remain conventional feedback controllers. Figure 1 shows the resulting stack: each layer runs faster than the one above it, and observations are fed back to every learned layer.

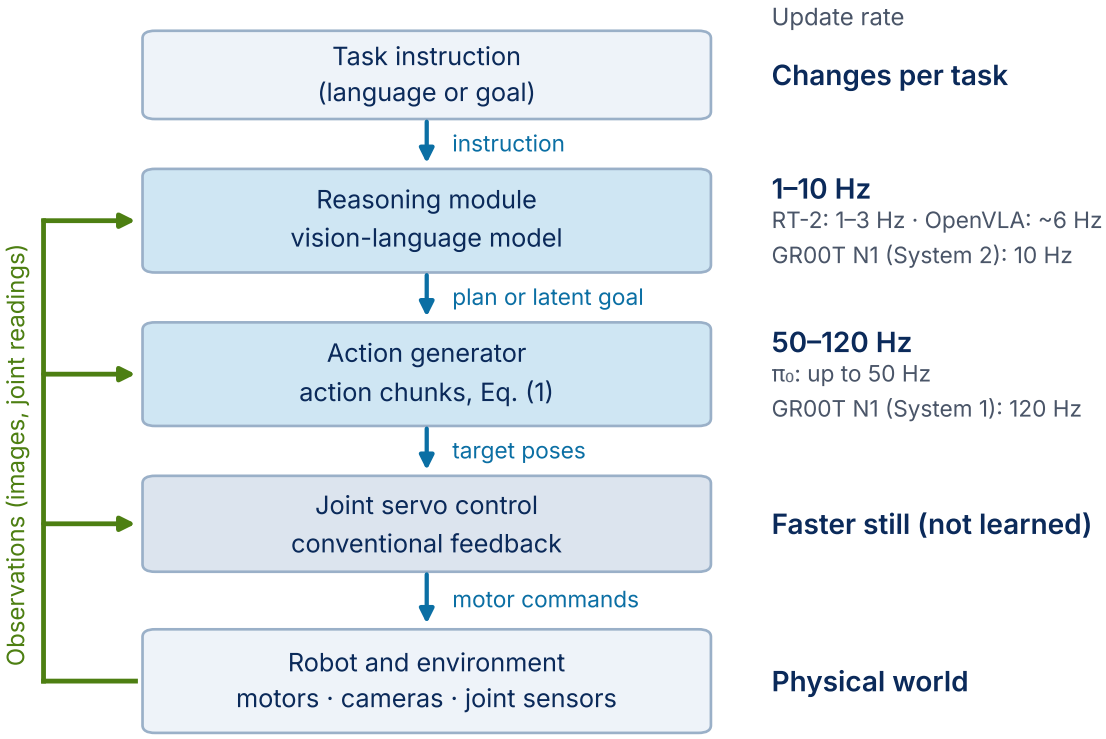



**Figure 1.** The perception, planning and control stack of a current robot learning system, with the update rate of each layer. Rates are those reported for the models named and were measured on different hardware.

Drawn by the author; rates from [7–10].

# 4. Learning Paradigms

## 4.1 Reinforcement learning

Reinforcement learning (RL) searches for a policy that maximizes the expected discounted sum of rewards collected through interaction:

$$J(\pi) = \mathrm{E}_{\pi}\left[ \sum_{t=0}^{T} \gamma^{t} \, r(s_t, a_t) \right] \tag{2}$$

where $s$ is the state, $r$ the reward, $\gamma \in (0, 1]$ the discount factor and $T$ the horizon. RL needs no demonstrations and can discover behaviors that exceed human skill, but it needs very large numbers of trials and a reward function that expresses the task without loopholes. Kober and co-authors identified the cost of real-world samples, the dimensionality of continuous states and actions, model error and reward specification as the main difficulties in robotics [3]. In

practice, RL for robots is now run almost entirely in simulation. Its clearest successes are in legged locomotion and in-hand manipulation, where rewards are easy to state and simulation is tractable (Sections 6 and 7).

## 4.2 Imitation learning and compounding errors

Imitation learning trains the policy by supervised learning on demonstrations, usually collected by a person teleoperating the robot. It avoids reward design and exploration, and it is the basis of most manipulation systems today. Its weakness is distribution shift. A policy trained only on expert trajectories has seen only states that the expert visits; once it makes a small mistake, it enters a state it was not trained on, where further mistakes are more likely. Ross and co-authors formalized the effect: if a behavior-cloned policy errs with probability ε per step on the expert's state distribution, the expected number of mistakes over a horizon of $T$ steps can grow as

$$\mathrm{E}[\text{mistakes}]_{\mathrm{BC}} \le \varepsilon T^{2} \text{ whereas } \mathrm{E}[\text{mistakes}]_{\text{interactive}} = O(\varepsilon T) \tag{3}$$

The quadratic term applies to plain behavior cloning (BC); the linear rate is achieved when the learner's own states are relabeled by the expert and added to the training set, as in the DAgger algorithm [11]. The practical lesson is that long tasks are much harder than short ones for a cloned policy, and that data showing recovery from errors are worth more than further perfect demonstrations. Figure 2 plots the two rates for ε = 0.01: over a 50-step task the bound for behavior cloning is 25 mistakes, against 0.5 for the linear rate.

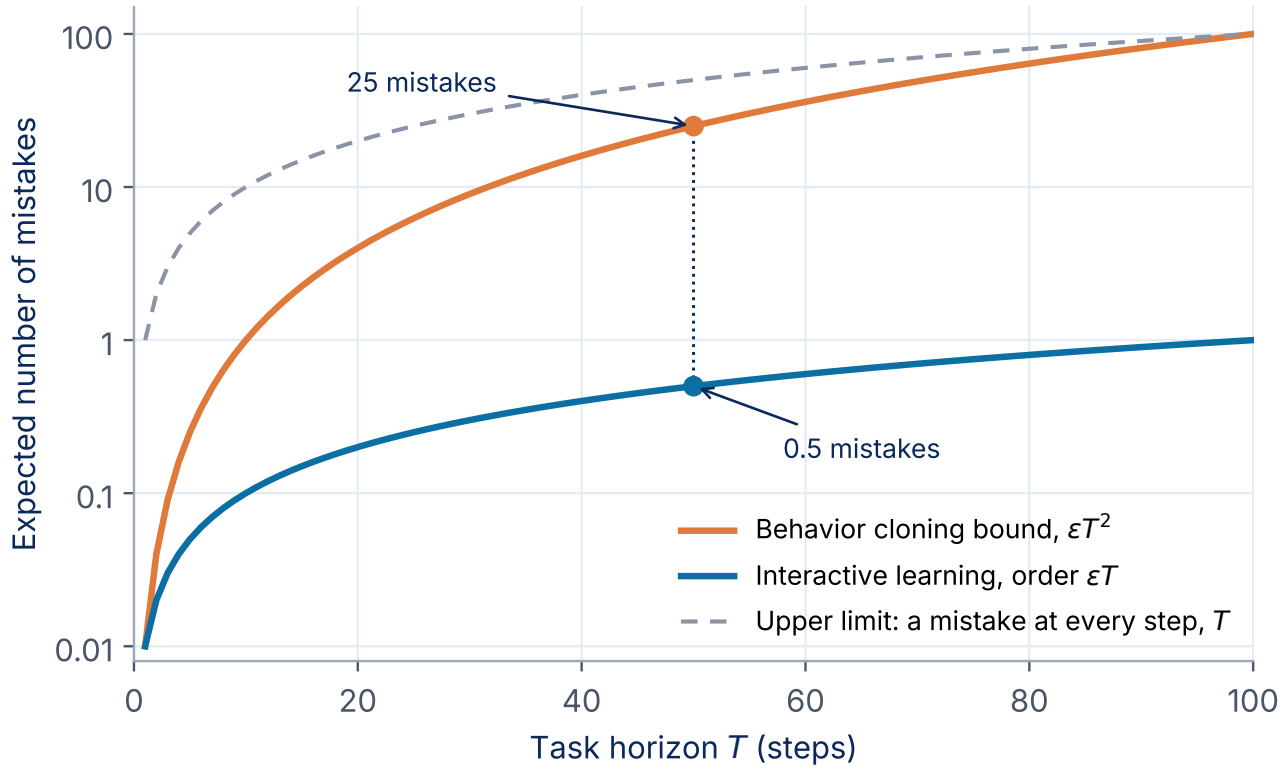



**Figure 2.** Growth of the expected number of mistakes with task horizon for a per-step error probability ε = 0.01, from Eq. (3). The behavior-cloning bound grows quadratically until it reaches the limit of one mistake per step at $T = 100$; with interactive relabeling the growth is linear. The vertical axis is logarithmic.

Author's calculation from Eq. (3), after [11].

### 4.3 Generative action models

Two techniques reduced the compounding-error problem without interactive relabeling. Action chunking, Eq. (1), shortens the effective horizon. With it, the ACT method learned six fine bimanual tasks, such as opening a translucent condiment cup and slotting a battery, at 80 to 90% success from about 10 minutes of demonstrations each, on low-cost hardware [6]. Diffusion Policy represents the policy as a conditional denoising diffusion process over action sequences, which captures multi-modal action distributions; across 12 tasks from four benchmarks it improved on prior methods by an average of 46.9% [12]. Both ideas, chunked prediction and generative modeling of actions, were carried into the foundation models described next.

## 5. Robot Foundation Models

## 5.1 From task-specific policies to generalist models

The policies in Section 4 are trained for one task on one robot. Foundation models aim for a single model that performs many tasks, follows language instructions and adapts to a new robot with little data. RT-1 established that a transformer policy improves with data scale and diversity: it was trained on about 130,000 demonstrations covering more than 700 instructions, collected with 13 robots over 17 months, and performed 97% of more than 200 seen instructions successfully [13]. RT-2 took the next step by fine-tuning a vision-language model on robot data together with web data. Each action dimension is discretized into 256 bins and written as text tokens, so that the model emits actions in the same way it emits words. Its authors named this class vision-language-action (VLA) models and reported, over about 6,000 evaluation trials, better generalization to new objects and to instructions absent from the robot data [7].

A different use of language models keeps them out of the control loop. In SayCan, a language model proposes the next step of a long task in words, and learned value functions for a library of pretrained skills score which proposals are feasible in the current scene, so that the plan is grounded in what the robot can do [14]. This planner-plus-skills pattern remains common in industry because each skill can be validated separately.

## 5.2 Data

Because no web-scale corpus of robot actions exists, the field pooled data. The Open X-Embodiment collaboration assembled datasets from 22 robot types contributed by 21 institutions, covering 527 skills, and showed that a model trained on the pooled data improved the performance of several robots relative to models trained on each robot's data alone [15]. DROID added scene diversity: 76,000 trajectories, or 350 hours, collected in 564 scenes by 50 operators over 12 months [16]. These numbers remain small by the standards of language modeling, and collecting them required years of coordinated human effort, which is why simulation and video are attractive additional sources. GR00T N1 is trained on what its authors call a data pyramid, with web data and human video at the base, synthetic data in the middle and real-robot data at the top [10].

## 5.3 Current models

Table 2 lists representative models. Octo is an open transformer policy trained on 800,000 trajectories from Open X-Embodiment that can be fine-tuned to new sensors and action

spaces within a few hours on consumer graphics hardware [17]. OpenVLA, trained on 970,000 demonstrations, outperformed the closed RT-2-X model by 16.5 percentage points in absolute success rate across 29 tasks with one-seventh of the parameters [8]. $\pi_0$ was pre-trained on more than 10,000 hours of data from seven robot configurations and was demonstrated on long, dexterous tasks such as folding laundry and assembling boxes [9]. These results show real progress in breadth. They do not yet show the reliability that industrial use requires: the OpenVLA authors state that success rates on their tested tasks are typically below 90% [8].

**Table 2.** Representative robot policies and foundation models, 2022 to 2025.

| Model | Year | Training data | Action generation | Reported rate |
|---|---|---|---|---|
| RT-1 [13] | 2022 | ~130,000 demonstrations, 13 robots | Discrete action tokens | 3 Hz |
| RT-2 [7] | 2023 | Robot data co-trained with web vision-language data | Actions as text tokens (256 bins) | 1–3 Hz (55B); ~5 Hz (5B) |
| Octo [17] | 2024 | 800,000 trajectories (Open X-Embodiment) | Diffusion action head | — |
| OpenVLA [8] | 2024 | 970,000 demonstrations | Actions as tokens, 7B parameters | ~6 Hz |
| $\pi_0$ [9] | 2024 | >10,000 hours, 7 robot configurations | Flow matching, action chunks | Up to 50 Hz |
| GR00T N1 [10] | 2025 | Real-robot, human video and synthetic data | Dual system: VLM plus diffusion transformer | 10 Hz and 120 Hz |

Rates are those reported by each paper on its own hardware and are not directly comparable.

# 6. Simulation and World Models

## 6.1 Physics simulators

A physics simulator integrates the equations of motion of articulated bodies with contact and returns synthetic sensor readings. The decisive change for learning was to run both the

simulation and the policy network on a graphics processor. Isaac Gym reported training times two to three orders of magnitude shorter than pipelines that simulate on a central processor [18], and with thousands of simulated robots in parallel on one processor, a walking policy for flat terrain was trained in under four minutes and transferred to a real quadruped [19]. Simulation at this speed converts the sample-inefficiency of RL from a barrier into a computing cost.

### 6.2 Learned world models

A world model is a learned predictor of how observations evolve under actions. Ha and Schmidhuber showed that an agent can learn a compressed model of its environment without supervision, train a compact policy entirely inside that model, and transfer the policy back to the real environment [20]. DreamerV3 developed the idea into a general algorithm that learns a model and improves its behavior by imagining future scenarios; with a single configuration it outperformed specialized methods on more than 150 tasks [21]. World foundation models extend the concept to video generators trained on large video corpora, which can be fine-tuned to a given robot and used to generate or augment training data; the Cosmos platform released such models with open weights for this purpose [1]. Learned world models complement physics simulators: they capture appearance and hard-to-model phenomena from data, but they offer no assurance that their predictions obey physical law, so policies trained inside them inherit the model's errors.

## 7. The Simulation-to-Reality Gap

### 7.1 Definition and sources

A policy that performs well in simulation often performs worse on hardware. The difference can be written as

$$G(\pi) = J_{\mathrm{sim}}(\pi) - J_{\mathrm{real}}(\pi) \tag{4}$$

where $J$ is the performance measure of Eq. (2) or a task success rate, evaluated in simulation and on the real system. The gap arises because the simulator is a model, and optimization exploits model error: a learning algorithm will find and use any inaccuracy that raises simulated reward [22]. The main sources are (i) dynamics, including friction, contact stiffness,

link masses and, above all, actuator behavior; (ii) timing, including sensing and actuation delays that are absent in an idealized simulation; (iii) perception, where rendered images differ from camera images in texture, lighting and noise; and (iv) unmodeled phenomena such as cable drag, backlash, deformable objects and wear. Actuators are a leading case. Hwangbo and co-authors replaced the analytic actuator model with a neural network trained on data from the real actuators, which allowed policies trained in simulation to run on the ANYmal robot [23].

## 7.2 Domain randomization

If the simulator cannot be made exact, the policy can be made insensitive to its errors. Domain randomization trains over a distribution of simulators instead of one:

$$\theta^{*} = \arg\max_{\theta} \; \mathbb{E}_{\xi \sim p(\xi)}\left[ J_{\xi}(\pi_{\theta}) \right] \tag{5}$$

where $\xi$ is a vector of simulator parameters, such as friction coefficients, masses, delays, textures and lighting, drawn from a distribution $p(\xi)$ at the start of each episode. Tobin and co-authors showed that an object detector trained only on simulated images with random, non-realistic textures localized real objects to within 1.5 cm, on the argument that with enough variability the real world appears to the model as one more variation [24]. Peng and co-authors applied the same idea to dynamics and obtained a pushing policy that kept its performance on a real arm without any training on hardware [25]. The method has a cost. If $p(\xi)$ is too narrow, the real system lies outside it and transfer fails; if it is too wide, the policy becomes conservative or the task becomes unlearnable [22]. Automatic domain randomization widens the distribution progressively as the policy improves, and was used to train a five-fingered hand to manipulate a Rubik's Cube [26]. Figure 3 illustrates the narrow, suitable and wide cases.

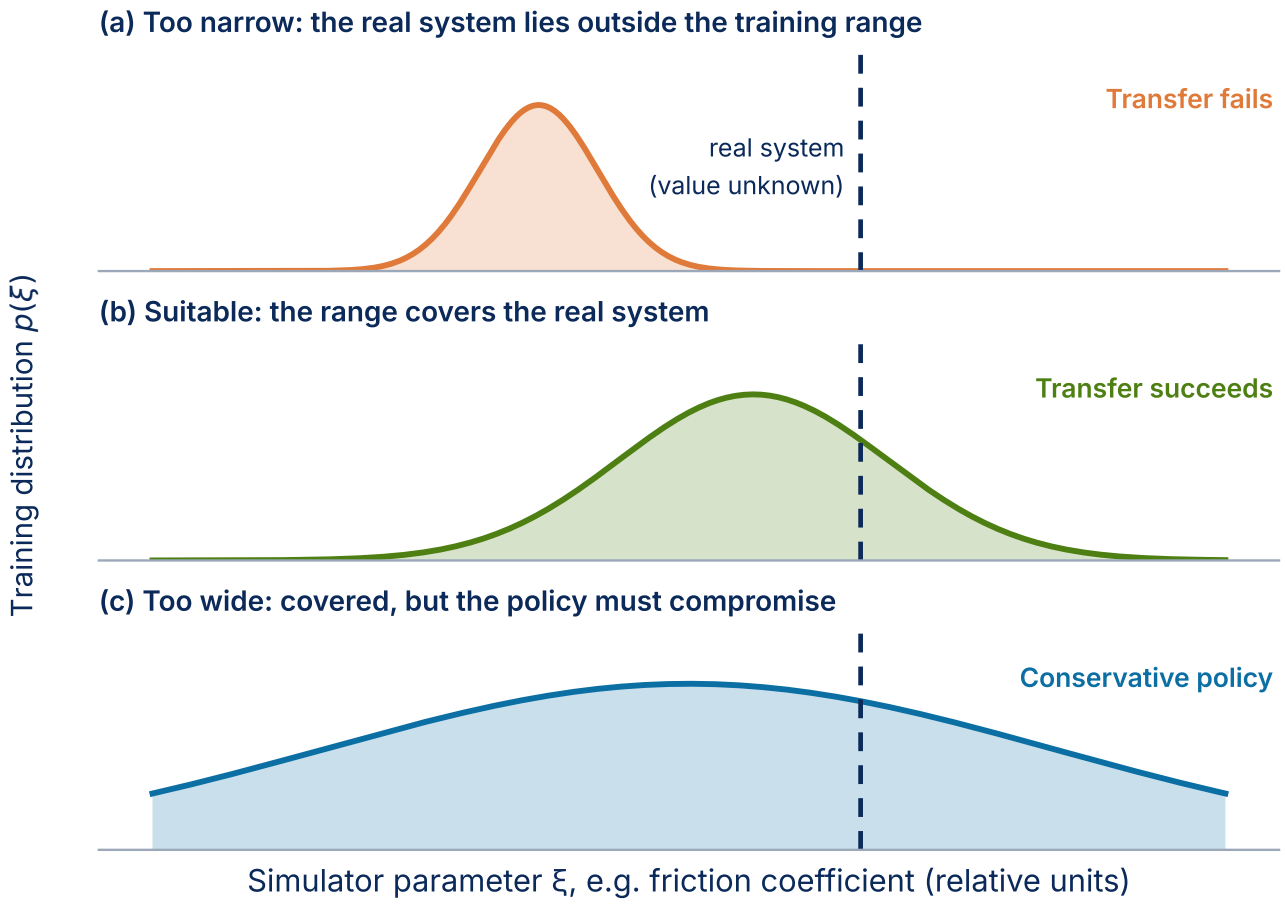



**Figure 3.** Domain randomization, Eq. (5), shown schematically for one simulator parameter. The policy is trained over a distribution $p(\xi)$ of simulator parameters; transfer depends on whether that distribution covers the unknown value of the real system without being so wide that the policy must compromise.

Schematic drawn by the author, after [22, 24, 25].

## 7.3 Adaptation, privileged learning and real data

A robust policy that ignores the parameters ξ must compromise across all of them. Better performance is available if the policy infers the conditions it is operating in. In privileged or teacher–student learning, a teacher policy is trained in simulation with access to ground-truth quantities such as terrain shape and friction, and a student policy is then trained to imitate the teacher using only the sensors available on the robot; a quadruped trained this way crossed mud, snow, rubble and running water that it had never encountered in training [27]. Rapid Motor Adaptation adds a module that estimates a compact description of the environment from the recent history of states and actions, and adapts within fractions of a second to changes in terrain and payload, again without fine-tuning on hardware [28]. For manipulation,

where contact and appearance are harder to simulate, the prevailing practice is to combine simulation with real demonstrations, or to pre-train broadly and fine-tune on a small amount of data from the target robot [17, 8]. Table 3 compares the methods.

**Table 3.** Methods for closing the simulation-to-reality gap.

| Method | Principle | Example | Main limitation |
|---|---|---|---|
| System identification | Make the simulator match the robot | Learned actuator model [23] | Effort per robot; some effects resist modeling |
| Domain randomization | Train over many simulators, Eq. (5) | Visual [24] and dynamics [25] randomization | Range must be chosen; wide ranges give conservative policies |
| Automatic randomization | Widen ranges as the policy improves | In-hand manipulation [26] | Very large compute budget |
| Privileged learning | Teacher sees ground truth; student uses real sensors | Locomotion on natural terrain [27] | Requires quantities observable in simulation |
| Online adaptation | Infer conditions from recent history | Rapid Motor Adaptation [28] | Adapts only within the training distribution |
| Real-data fine-tuning | Correct residual error with target-robot data | Fine-tuning generalist policies [17, 8] | Needs demonstrations on each new setup |

Whether simulation alone can scale to general manipulation remains debated. A published discussion among researchers in the field recorded both positions: that sufficiently randomized simulation can cover reality, and that simulators will not capture contact and deformable materials well enough, so that real data remain necessary [29]. The evidence so far is that simulation-only training succeeds where the physics is dominated by rigid bodies and the reward is simple, and that tasks with rich contact and varied objects still need real data.

The simulation-to-reality gap is a model-error problem. It is narrowed by better models where the physics is known, by randomization where it is uncertain, and by real data where it cannot be modeled.

# 8. Evaluation and Open Problems

## 8.1 Evaluation

Results in this field are reported as success rates over physical trials, and three cautions apply when reading them. First, trial counts are small, so uncertainty is large: a success rate measured over 20 trials has a 95% confidence interval of about ±22 percentage points near 50% success. Second, results are specific to the robot, scene and objects tested, and the control rates in Table 2 were measured on different hardware. Third, a success rate that is impressive for research may be far from adequate in service. Section 4.2 showed that errors compound over the length of a task, and a companion paper in this volume treats the reliability levels that industrial deployment requires.

## 8.2 Open problems

Four problems limit current systems. The first is reliability over long horizons, for the reason just given. The second is distribution shift after deployment. Learned components can fail without warning on inputs unlike their training data, and Sinha and co-authors argue that this must be handled at the level of the whole system, with monitoring to detect such inputs and fallback behaviors when they are detected [30]. The third is data: teleoperated demonstrations cost operator time on real robots, and it is not yet established how far video and simulation can substitute for them [4]. The fourth is computation at the robot. Large models run at a few hertz on a graphics processor, as Section 3.3 showed, so that fast, contact-rich skills need either smaller action modules or hierarchical designs.

- Use simulation-trained RL where dynamics are rigid-body and the objective is simple, as in locomotion.
- Use imitation learning with chunked, generative action models for manipulation, and collect demonstrations that include recovery from errors.

- Treat a foundation model as a starting point to be fine-tuned and validated on the target robot and task, not as a finished controller.
- Measure the reality gap, Eq. (4), on the real system before relying on any simulated result.

## 9. Conclusion

Physical AI applies learned decision-making to systems that sense and act in the material world. Its methods are now well defined: reinforcement learning in fast parallel simulation, imitation learning with generative action models, and vision-language-action models that bring web-scale knowledge to robot control. Its central difficulty is also well defined. Robot experience is expensive, so systems are trained largely on substitutes for it, in simulators, on video and on pooled data from other robots, and each substitute introduces a gap that must be measured and closed on the target system. The techniques reviewed here, including system identification, domain randomization, privileged learning, online adaptation and fine-tuning on real data, have closed that gap for locomotion and for a growing set of manipulation tasks. The remaining distance to dependable general-purpose machines is mainly one of reliability over long tasks and under conditions not seen in training. For engineers assessing the technology, the appropriate questions are therefore how a system was trained, on what evidence its performance on the real task rests, and what it does when it meets a situation outside its experience.

## References


1. NVIDIA, Agarwal, N., Ali, A., Bala, M., Balaji, Y., Barker, E., Cai, T., Chattopadhyay, P., Chen, Y., et al. (2025). Cosmos world foundation model platform for physical AI. arXiv preprint arXiv:2501.03575. arxiv.org/abs/2501.03575
2. Brooks, R. A. (1991). Intelligence without representation. Artificial Intelligence, 47(1–3), 139–159. 10.1016/0004-3702(91)90053-M
3. Kober, J., Bagnell, J. A., & Peters, J. (2013). Reinforcement learning in robotics: A survey. The International Journal of Robotics Research, 32(11), 1238–1274. 10.1177/0278364913495721
4. Firoozi, R., Tucker, J., Tian, S., Majumdar, A., Sun, J., Liu, W., Zhu, Y., Song, S., Kapoor, A., Hausman, K., et al. (2023). Foundation models in robotics: Applications, challenges, and the future. arXiv preprint arXiv:2312.07843. arxiv.org/abs/2312.07843

5. Levine, S., Finn, C., Darrell, T., & Abbeel, P. (2016). End-to-end training of deep visuomotor policies. Journal of Machine Learning Research, 17(39), 1–40. arxiv.org/abs/1504.00702
6. Zhao, T. Z., Kumar, V., Levine, S., & Finn, C. (2023). Learning fine-grained bimanual manipulation with low-cost hardware. arXiv preprint arXiv:2304.13705. arxiv.org/abs/2304.13705
7. Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Chen, X., Choromanski, K., Ding, T., Driess, D., Dubey, A., Finn, C., et al. (2023). RT-2: Vision-language-action models transfer web knowledge to robotic control. arXiv preprint arXiv:2307.15818. arxiv.org/abs/2307.15818
8. Kim, M. J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E., Lam, G., Sanketi, P., et al. (2024). OpenVLA: An open-source vision-language-action model. arXiv preprint arXiv:2406.09246. arxiv.org/abs/2406.09246
9. Black, K., Brown, N., Driess, D., Esmail, A., Equi, M., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., et al. (2024). $\pi_0$: A vision-language-action flow model for general robot control. arXiv preprint arXiv:2410.24164. arxiv.org/abs/2410.24164
10. NVIDIA, Bjorck, J., Castañeda, F., Cherniadev, N., Da, X., Ding, R., Fan, L., Fang, Y., Fox, D., et al. (2025). GR00T N1: An open foundation model for generalist humanoid robots. arXiv preprint arXiv:2503.14734. arxiv.org/abs/2503.14734
11. Ross, S., Gordon, G. J., & Bagnell, J. A. (2011). A reduction of imitation learning and structured prediction to no-regret online learning. Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS), PMLR 15, 627–635. arxiv.org/abs/1011.0686
12. Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., & Song, S. (2023). Diffusion Policy: Visuomotor policy learning via action diffusion. arXiv preprint arXiv:2303.04137. arxiv.org/abs/2303.04137
13. Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Dabis, J., Finn, C., Gopalakrishnan, K., Hausman, K., Herzog, A., Hsu, J., et al. (2022). RT-1: Robotics Transformer for real-world control at scale. arXiv preprint arXiv:2212.06817. arxiv.org/abs/2212.06817
14. Ahn, M., Brohan, A., Brown, N., Chebotar, Y., Cortes, O., David, B., Finn, C., Fu, C., Gopalakrishnan, K., Hausman, K., et al. (2022). Do as I can, not as I say: Grounding language in robotic affordances. arXiv preprint arXiv:2204.01691. arxiv.org/abs/2204.01691
15. Open X-Embodiment Collaboration, O'Neill, A., Rehman, A., Gupta, A., Maddukuri, A., Gupta, A., Padalkar, A., Lee, A., Pooley, A., Gupta, A., et al. (2023). Open X-Embodiment: Robotic learning datasets and RT-X models. arXiv preprint arXiv:2310.08864. arxiv.org/abs/2310.08864
16. Khazatsky, A., Pertsch, K., Nair, S., Balakrishna, A., Dasari, S., Karamcheti, S., Nasiriany, S., Srirama, M. K., Chen, L. Y., Ellis, K., et al. (2024). DROID: A large-scale in-the-wild robot

manipulation dataset. arXiv preprint arXiv:2403.12945. arxiv.org/abs/2403.12945

17. Octo Model Team, Ghosh, D., Walke, H., Pertsch, K., Black, K., Mees, O., Dasari, S., Hejna, J., Kreiman, T., Xu, C., et al. (2024). Octo: An open-source generalist robot policy. arXiv preprint arXiv:2405.12213. arxiv.org/abs/2405.12213
18. Makoviychuk, V., Wawrzyniak, L., Guo, Y., Lu, M., Storey, K., Macklin, M., Hoeller, D., Rudin, N., Allshire, A., Handa, A., & State, G. (2021). Isaac Gym: High performance GPU-based physics simulation for robot learning. arXiv preprint arXiv:2108.10470. arxiv.org/abs/2108.10470
19. Rudin, N., Hoeller, D., Reist, P., & Hutter, M. (2021). Learning to walk in minutes using massively parallel deep reinforcement learning. arXiv preprint arXiv:2109.11978. arxiv.org/abs/2109.11978
20. Ha, D., & Schmidhuber, J. (2018). World models. arXiv preprint arXiv:1803.10122. arxiv.org/abs/1803.10122
21. Hafner, D., Pasukonis, J., Ba, J., & Lillicrap, T. (2025). Mastering diverse control tasks through world models. Nature, 640(8059), 647–653. 10.1038/s41586-025-08744-2
22. Zhao, W., Queralta, J. P., & Westerlund, T. (2020). Sim-to-real transfer in deep reinforcement learning for robotics: A survey. 2020 IEEE Symposium Series on Computational Intelligence (SSCI), 737–744. 10.1109/SSCI47803.2020.9308468
23. Hwangbo, J., Lee, J., Dosovitskiy, A., Bellicoso, D., Tsounis, V., Koltun, V., & Hutter, M. (2019). Learning agile and dynamic motor skills for legged robots. Science Robotics, 4(26), eaau5872. 10.1126/scirobotics.aau5872
24. Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., & Abbeel, P. (2017). Domain randomization for transferring deep neural networks from simulation to the real world. arXiv preprint arXiv:1703.06907. arxiv.org/abs/1703.06907
25. Peng, X. B., Andrychowicz, M., Zaremba, W., & Abbeel, P. (2018). Sim-to-real transfer of robotic control with dynamics randomization. 2018 IEEE International Conference on Robotics and Automation (ICRA), 3803–3810. 10.1109/ICRA.2018.8460528
26. OpenAI, Akkaya, I., Andrychowicz, M., Chociej, M., Litwin, M., McGrew, B., Petron, A., Paino, A., Plappert, M., Powell, G., et al. (2019). Solving Rubik's Cube with a robot hand. arXiv preprint arXiv:1910.07113. arxiv.org/abs/1910.07113
27. Lee, J., Hwangbo, J., Wellhausen, L., Koltun, V., & Hutter, M. (2020). Learning quadrupedal locomotion over challenging terrain. Science Robotics, 5(47), eabc5986. 10.1126/scirobotics.abc5986
28. Kumar, A., Fu, Z., Pathak, D., & Malik, J. (2021). RMA: Rapid motor adaptation for legged robots. arXiv preprint arXiv:2107.04034. arxiv.org/abs/2107.04034

29. Höfer, S., Bekris, K., Handa, A., Gamboa, J. C., Mozifian, M., Golemo, F., Atkeson, C., Fox, D., Goldberg, K., Leonard, J., Liu, C. K., Peters, J., Song, S., Welinder, P., & White, M. (2021). Sim2Real in robotics and automation: Applications and challenges. IEEE Transactions on Automation Science and Engineering, 18(2), 398–400. 10.1109/TASE.2021.3064065

30. Sinha, R., Sharma, A., Banerjee, S., Lew, T., Luo, R., Richards, S. M., Sun, Y., Schmerling, E., & Pavone, M. (2022). A system-level view on out-of-distribution data in robotics. arXiv preprint arXiv:2212.14020. arxiv.org/abs/2212.14020

### How to cite this article

Lee, Q. F. (2026). Physical AI: Foundations, Architectures and the Simulation-to-Reality Gap. *Chmltech Online Journal*, *105*(1). https://www.chmltech.com/post/physical-ai-foundations-architectures-and-the-sim-to-real-gap

## Ready to find your next improvement?

Book a strategy call. We will talk through your goals, pain points and where AI, systems or process work can make the biggest difference.

Book a Strategy Call

Or email **info@chmltech.com**